

RTCM: A Natural Language Based, Automated, and Practical Test Case Generation Framework

Tao Yue
Simula Research Laboratory and
University of Oslo
Oslo, Norway
+47 40433125
tao@simula.no

Shaukat Ali
Simula Research Laboratory
Oslo, Norway
+47 47466831
shaukat@simula.no

Man Zhang
Simula Research Laboratory
Oslo, Norway
manzhang@simula.no

ABSTRACT

Based on our experience of collaborating with industry, we observed that test case generation usually relies on test case specifications (TCSs), commonly written in natural language, specifying test cases of a System Under Test at a high level of abstraction. In practice, TCSs are commonly used by test engineers as reference documents to perform these activities: 1) Manually executing test cases in TCSs; 2) Manually coding test cases in a test scripting language for automated test case execution. In the latter case, the gap between TCSs and executable test cases has to be filled by test engineers, requiring a significant amount of coding effort and domain knowledge. Motivated by the above observations from the industry, we first propose, in this paper, a TCS language, named as Restricted Test Case Modeling (RTCM), which is based on natural language and composed of an easy-to-use template, a set of restriction rules and keywords. Second, we propose a test case generation tool (aToucan4Test), which takes TCSs in RTCM as input and generates either manual test cases or automatically executable test cases, based on various coverage criteria defined on RTCM. To assess the applicability of RTCM, we manually modeled two industrial case studies and examined 30 automatically generated TCSs. To evaluate aToucan4Test, we modeled three subsystems of a Video Conferencing System developed by Cisco Systems, Norway and automatically generated executable test cases. These test cases were successfully executed on two commercial software versions. In the paper, we also discuss our experience of applying RTCM and aToucan4Test in an industrial context and compare our approach with other model-based testing methodologies.

Categories and Subject Descriptors

D.2.5 [Software Engineering]: Testing and Debugging—*testing tools*.

General Terms

Documentation, Design, Experimentation, Languages.

Keywords

RUCM; Test Case Specification; Test Case Generation; Model Based Testing; Model Transformation; Natural Language.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from Permissions@acm.org.

ISSTA '15, July 13–17, 2015, Baltimore, MD, USA
© 2015 ACM. 978-1-4503-3620-8/15/07...\$15.00
<http://dx.doi.org/10.1145/2771783.2771799>

1. INTRODUCTION

Test case specifications (TCSs) are a common way of documenting test cases for a System Under Test (SUT) [20]. Test engineers use such specifications to either execute documented test cases manually or code them in a test scripting language for automated testing. A common practice is to write TCSs in natural language (NL), which requires a significant amount of coding effort to write syntactically correct and automatically executable test cases. Moreover, manual coding of test cases heavily relies on the experience and tacit knowledge of test engineers and therefore there is no systematic way of ensuring test coverage at the requirements level.

In this paper, we aim to facilitate the transition from textual TCSs to executable test cases by developing a TCS language called Restricted Test Case Modeling (RTCM), and an automated test case generation tool (named as aToucan4Test). RTCM combines NL with a set of restriction rules, a set of keywords, and an easy to use template for writing TCSs. All RTCM concepts are formalized in a metamodel named as *TCMeta*. RTCM and TCMeta are built on the top of the Restricted Use Case Modeling (RUCM) approach and its corresponding use case metamodel (UCMeta), respectively. It is worth mentioning that RUCM and UCMeta were proposed to enable the automated generation of UML analysis models, which is a different objective as RTCM and TCMeta [34; 35].

RTCM can be used to specify TCSs with (or without) Application Programming Interfaces (APIs) information embedded, which can be automatically formalized as instances of TCMeta. The generated TCMeta instances can then be automatically transformed into either manual test cases or automatically executable test cases. It is also important to mention that in our previous work [37], we have proposed an approach to transform requirements (specified in RUCM) into RTCM TCSs without API information.

To compare with the related work and our previous work, the main contributions of this paper include: 1) An easy to use template for specifying TCSs, combining NL with restrictions and keywords (e.g., keywords related to test configuration and stimulus to a SUT via APIs); 2) Formalization of RTCM as *TCMeta* to facilitate automation; 3) Test generation tool called aToucan4Test that implements various test coverage criteria on RTCM to generate automatically executable test cases.

To assess the applicability of RTCM, we manually specified (using RTCM) two industrial case studies from two different industry domains: tele-communication and oil&gas. In addition, we examined other 30 TCSs in RTCM that were automatically generated from use case models by an approach proposed in our previous work [37]. Results show that RTCM notations are

sufficient to specify TCSs from diverse case studies. To evaluate aToucan4Test, we modeled a commercial Video Conferencing System (VCS), called Saturn developed by Cisco Systems Norway [2]. In total we modeled nine specifications corresponding to three subsystems of Saturn. With our tool, aToucan4Test we generated 246 syntactically correct and executable test cases from the nine TCSs. These test cases were successfully automatically executed to test two versions of software, indicating that aToucan4Test can generate syntactically correct test cases. In the paper, we also report our experience of applying RTCM and aToucan4Test for the industrial case study and how it positions itself among other Model-Based Testing (MBT) approaches.

The rest of the paper is organized as follows: Section 2 presents the background necessary to understand the paper. The related work is presented in Section 3. The RTCM modeling methodology is described in Section 4, followed by aToucan4Test (Section 5). Section 6 discusses the evaluation of RTCM, followed by our industrial experience (Section 7). We conclude the paper in Section 8.

2. BACKGROUND

In our previous work, we have devised the RUCM methodology, which encompasses a use case template and 26 restriction rules for the textual use case specifications [35]. The goal of RUCM is to be easy to use, to reduce ambiguity and improve understanding, and to facilitate automated analysis. We therefore performed a controlled experiment to evaluate RUCM in terms of its ease of use and the quality of manually derived analysis models [34; 35]. Results showed that RUCM is overall easy to use, and that it results in significant improvements over the use of a standard use case template [12] in terms of the understandability of use case specifications.

A RUCM use case specification has one basic flow and can have one or more alternative flows. An alternative flow always depends on a condition occurring in a specific step in a flow of reference, referred to as reference flow, which is either the basic flow or an alternative flow itself. We classify alternative flows into three types: A specific alternative flow refers to a specific step in the reference flow; a bounded alternative flow refers to more than one step (consecutive or not) in the reference flow; a global alternative flow (called general alternative flow in [9]) refers to any step in the reference flow. The different types of alternative flows specify precisely the interactions between the reference flow and its alternative flows. For specific and bounded alternative flows, a RFS (Reference Flow Step) section specifies one or more (reference flow) step numbers.

The 26 restriction rules of RUCM are classified into two categories: 1) restrictions on the use of NL (16 rules), and 2) rules enforcing the use of keywords for specifying control structures (10 rules). In the second category, RUCM defines a set of keywords to specify conditional logic sentences (IF-THEN-ELSE-ELSEIF-ENDIF), concurrency sentences (MEANWHILE), condition checking sentences (VALIDATES THAT), and iteration sentences (DO-UNTIL). These keywords limit opportunities for ambiguities in use case specifications. They also greatly facilitate the automated generation of other models such as UML sequence diagrams.

UCMeta is a metamodel that can be used to formalize textual RUCM to facilitate such as the automated generation of UML analysis models from RUCM models [17; 34]. UCMeta is specified using Meta-Object Facility (MOF) [28] and the

formalization of RUCM models to UCMeta instances are done automatically, as described in [34]. In this paper, we propose a new metamodel named as TCMeta, by extending UCMeta, to formalize RTCM specifications (see Section 4 for details).

3. RELATED WORK

We discuss the related work from three aspects: Requirements based testing, keywords driven testing and Behavior Driven Development (BDD).

3.1 Requirements Based Testing

A controlled NL, named as uscCNL, was proposed in [7] for use case specifications with the objective of facilitating automated generation of test cases. To compare with RUCM, the proposed method does not include a use case template, which is commonly required when documenting a use case specification and considered as an important means to structure textual requirements. RUCM however includes a comprehensive, and well-evaluated use case template, which is also the foundation for structuring RTCM TCSs. In addition, both RUCM and RTCM rely on the keyword mechanism (as part of the restriction rules) to enhance the precision of specifications and enable automation. uscCNL, however defines a quite comprehensive list of grammars for using English. Notice that RUCM/RTCM specifications can also be automatically formalized as instances of UCMeta/TCMeta. Therefore the identification of English sentence patterns and grammar can be automated.

In [6], authors have proposed a methodology to generate test cases using use cases and collaboration diagrams. Their process starts with constructing use cases for the system, which are realized by constructing a sequence diagram corresponding to each use case. In order to manipulate the collaboration diagram for test case generation, a customized formal language known as Collaboration Description Language was used. This language represents description of the collaboration diagram in a textual format. Our approach directly takes RTCM specifications as input to automatically generate executable test cases. No additional modeling notations are required in between.

In [24], authors have proposed a methodology to generate test cases for system testing by taking use cases with associated contracts and parameters as input. First the use cases are parameterized, i.e., the inputs to the use cases are defined. Then the contracts including pre and post conditions of the use cases are defined. Pre and post conditions are defined in the form of logical expressions. The relationships between the use cases are also defined within logical expressions. The mappings of include, exclude, and extend relationships to logical expressions were also defined. Finally test cases were generated by instantiating the use case parameters. Notice that no detailed steps of use case specifications are exercised to generate test cases in their approach. We however generate test cases by taking into account flows of events and their steps of each TCS while applying different coverage criteria.

In [29] authors have proposed an approach to generate test cases for system testing from scenarios, which represent requirements in NL. Therefore, scenarios tend to be more ambiguous, so a formal model was needed into which scenarios are converted. Test cases are then generated from the generated formal model. In this approach state machines were used as converted models to generate test cases. RTCM is rigorous to compare with free style NL. Moreover, we have an automated solution to formalize RUCM specifications as instances of TCMeta.

Tahan et al. [30] proposed a requirements based, black box test generation approach to generate test cases from requirements specified using System Description Language (SDL) [1; 10; 11; 15]. Each individual SDL requirement is first automatically transformed into a SDL system model. Manual combination of individual SDL system models to an integrated one is required. The combined SDL model is then automatically transformed into Extended Finite State Machine (EFSM), which is provided as input to a test generator for generating test cases. To compare with this approach, which takes three transformation steps (with manual intervention in between), our approach provides a direct transition from RTCM specifications to executable test cases.

Our previous work [37] proposes an automated solution to generate TCSs (in RTCM) from use case models specified in RUCM. The transformation aims to enable automated requirements-based testing and establish automated traceability between requirements and TCSs. However, it is important to mention that in this paper, we propose aToucan4Test to bridge the gap between TCSs in RTCM and executable test cases. Though in [37], RTCM was only used as the means to represent the output of the transformation: TCSs, but it was not discussed as the means to enable the automated generation of either manually or automatically executable test cases.

3.2 Keyword Driven Testing

Tang et al. [31] proposed a keyword driven testing framework to support the conversion of the keyword-based test cases into different kinds of test scripts. Keywords are used to identify operations or atomic actions in test execution and they are very specific to applications such as ‘Click’ for GUI applications. Test engineers use these keywords (via a GUI) to specify test cases, which are then transformed into test scripts. Notice that each test case defines a sequence of actions or operations. Therefore, there is no branch and loop in each test case description. Our RTCM however enables the definition of more complicated TCSs corresponding to multiple test cases, which is at a higher level of abstraction than their proposed approach.

Hametner et al. [18] proposed a keyword driven testing approach for industrial automation systems, with the aim to specify test cases in a high abstraction level and enable non-programmers in reading and writing test cases. Test engineers write test cases manually in a tabular format using the predefined keywords and automated generation of executable tests from the tabular format is not supported. Suresh et al. [32] proposed a technique to generate keyword-based scripts, which consist of sequences of tuples (*action-target-data*), from manual test cases written in natural language, and implemented a tool based natural language processing. Little and Miller [21] also proposed an automated approach to translate keyword commands into executable code, aiming to relief users from writing test scripts.

3.3 Behavior Driven Development

BDD [13] is a software development process based on Test Driven Development (TDD) [8], which describes the cycle of writing a test first before writing the code being tested. For TDD, to write a test, the developer must clearly understand the requirements

specification of a feature to be implemented. Such a requirements specification is often written as use cases and user stories, which essentially describes the behavior of the feature. BDD however concentrates on specifying desired behavior (e.g., business requirements) of SUT, which lead to the derivation of tests for testing SUT.

BDD chooses to use a semi-formal format for specifying behavior, which is similar to user story specifications. During the process, BDD specifies that business analysts and developers should collaborate and specify behavior in terms of user stories. Same as our approach, BDD aims to become a communication medium among all stakeholders in a project. Some templates have been proposed to specify user stories. For example, Dan North [27] suggested a textual format template including keywords like ‘in order to’, ‘given’ and ‘when’. Based on this textual format, Dan North proposed a number of frameworks that support BDD, including JBehave [25] for Java and RBehave [26] for Ruby, based on which Cucumber[14] was later on proposed. There also exist other BDD-based tools (e.g., Fitnesse [16; 22]), which use different formats such as decision tables. All these BDD tools do not support the automated generation of executable test cases like what we do. They instead aim to achieve agile development of software applications, including facilitating design, specifying behavior and documenting code. Our framework however can generate test cases.

4. RTCM MODELING METHODOLOGY

We present the RTCM modeling methodology using a simplified subsystem called Saturn of a VCS developed by Cisco System Norway as the running example. Detailed description of the system can be found in [2]. The core functionality of a VCS is sending and receiving multimedia streams. Saturn deals with establishing video conferencing calls, disconnecting calls, and starting/stopping presentation. It can also receive requests for establishing calls, disconnecting calls, and starting/stopping presentation from other VCSs (Endpoints) participating in a videoconference.

4.1 RTCM Template

RTCM is inspired by RUCM in terms of specifying scenarios. As shown in Figure 1, the essential idea of *ScenarioSpecification* is that a flow of events is composed of a sequence of steps (*Sentences*) and a *PostCondition*.

Flows of events are further classified into *BasicFlow* and

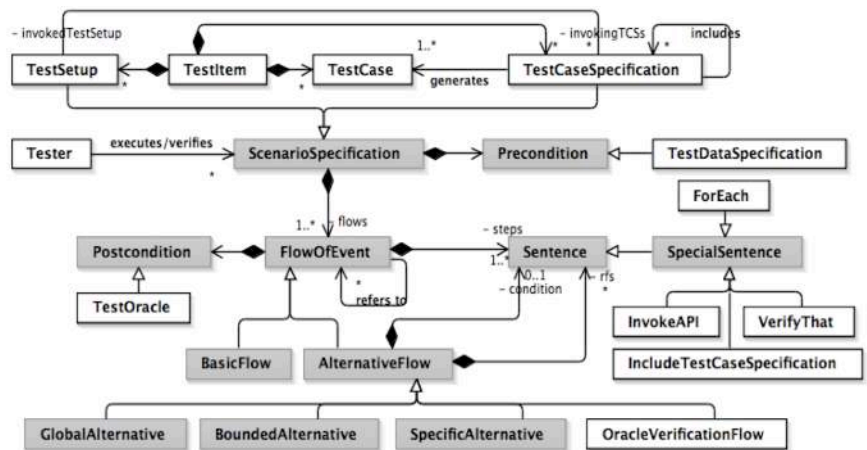


Figure 1. The Domain Model of RTCM

Test Case Specification		
Name	CallBehavior	
Brief Description	The test specification aims to test the behavior of the system under test makes calls to other endpoint.	
Precondition (Test Data Specification)	The device A Configuration.NetworkServices.H323.Mode=0n The device A Configuration.Conference.AutoAnswer.Mode=0n The device B Configuration.NetworkServices.H323.Mode=0n The device B Configuration.Conference.AutoAnswer.Mode=0n	
Tester	None	
Dependency	None	
Test Setup	Name	Create Device
▼	Description	This test setup aims to create an device to start a call behavior.
Basic Flow (Test Setup) (Untitled) ▼	Steps	
	1	The test system creates a device as A (IP=192.168.0.1,name=spa,username=admin,password=password,rootpassword=password).
	2	The test system creates a device as B (IP=192.168.0.2,name=spa,username=admin,password=password,rootpassword=password).
	Postcondition (Test Oracle)	The device A has been created.The device B has been created.
Basic Flow (Test Sequence) (Untitled) ▼	Steps	
	1	The test system VALIDATES THAT the device A Status.Conference.Presentation.Mode==Off.
	2	The test system VALIDATES THAT the device A Status.SystemUnit.State.NumberOfActiveCalls==0.
	3	DO
	4	The device A INVOKES API Command.Dial(the device B) to make a call.
	5	UNTIL the device A Status.SystemUnit.State.NumberOfActiveCalls > 0
	6	The device A INVOKES API Command.Presentation.Start() to start a presentation.
	7	The test system VALIDATES THAT the device A Status.Conference.Presentation.Mode == Sending.
	8	The device A INVOKES API Command.Presentation.Stop() to stop a presentation.
	9	The test system VALIDATES THAT the device A Status.Conference.Presentation.Mode == Off.
	10	The device A INVOKES API Command.Call.DisconnectAll() to disconnect all endpoints.
	Postcondition (Test Oracle)	The device A Status.Conference.Presentation.Mode == Off The device A Status.SystemUnit.State.NumberOfActiveCalls == 0
Specific Alt. Flow (Test Sequence) "TS1" ▼	RFS 1	
	1	The device A INVOKES API Command.Presentation.Stop() to stop the active presentation.
	2	RESUME STEP 2
	Postcondition (Test Oracle)	The device A Status.Conference.Presentation.Mode==Off
Specific Alt. Flow (Test Sequence) "TS2" ▼	RFS 2	
	1	The device A INVOKES API Command.Call.DisconnectAll() to end the active calls.
	2	RESUME STEP 3
	Postcondition (Test Oracle)	The device A Status.SystemUnit.State.NumberOfActiveCalls == 0
Oracle Verification Flow "O1" ▼	RFS 4	
	1	The test system VALIDATES THAT the device A Status.SystemUnit.State.NumberOfActiveCalls == @Pre The device A Status.SystemUnit.State.NumberOfActiveCalls + 1
	Postcondition (Test Oracle)	None
Oracle Verification Flow "O2" ▼	RFS 4	
	1	Tester VERIFIES THAT the recent called number is in the call history.
	Postcondition (Test Oracle)	None
Specific Alt. Flow (Test Sequence) "TS3" ▼	RFS 7	
	1	The device A INVOKES API Command.Presentation.Start() to start a presentation.
	2	RESUME STEP 8.
	Postcondition (Test Oracle)	The device A Status.Conference.Presentation.Mode == Sending
Specific Alt. Flow (Test Sequence) "TS4" ▼	RFS 9	
	1	The device A INVOKES API Command.Presentation.Stop() to stop a presentation.
	2	RESUME STEP 10.
	Postcondition (Test Oracle)	The device A Status.Conference.Presentation.Mode == Off

Figure 2. Test Case Specification for CallBehavior (editor in the RTCM editor)

AlternativeFlow, which is further classified into four types: *SpecificAlternative*, *BoundedAlternative*, *GlobalAlternative* and *OracleVerificationFlow*. *OracleVerificationFlow* is newly proposed for RTCM and the other three ones are from RUCM.

TestItem is a concept borrowed from the ISO/IEC/IEEE 29119 standard [20], where *TestItem* (an alternate to commonly used term SUT) is defined as “work product that is an object of testing”. The test item is composed of a set of *TestSetup* and *TestCaseSpecification*, both of which should be specified using

the *ScenarioSpecification* template. *TestSetup* can be shared across TCSs. We define *TestSetup* as a set of steps to get the test item ready for executing flows of events defined in a TCS. *TestCaseSpecification*, as defined in [20], is “documentation of a set of one or more test cases”.

A test engineer interacts with a test setup or TCS by either manually performing or verifying steps specified in the test

setup or the TCS. In both cases, it is impossible to execute test cases on the test item and hence steps specified in either the test setup or the TCS have to be run manually, which is very common in many domains involving both software and (mechanical and electrical) hardware components.

One TCS can be used to derive one or more test cases. Since the scenario specification template provides a nice way of

CallBehavior

Test Case Specification	
Name	CallBehavior
Brief Description	The test specification aims to test the behavior of the system under test makes calls to other endpoint.
Precondition (Test Data Specification)	The device A Configuration.NetworkServices.H323.Mode=On The device A Configuration.Conference.AutoAnswer.Mode=On The device B Configuration.NetworkServices.H323.Mode=On The device B Configuration.Conference.AutoAnswer.Mode=On
Tester	None
Dependency	None

Test Setup	Name	Description
▼	Create Device	This test setup aims to create an device to start a call behavior.

Basic Flow (Test Sequence)	Steps
(Unlabeled) ▼	1 The test system creates a device as A (IP=192.168.0.1,name=spa,username=admin,password=password,rootpassword=password).
	2 The test system creates a device as B (IP=192.168.0.2,name=sba,username=admin,password=password,rootpassword=password).
	3 Postcondition The device A has been created.The device B has been created.
	4 DO
	5 UNTIL the device A Status.SystemUnit.State.NumberOfActiveCalls > 0
	6 The device A INVOKES API Command.Presentation.Start() to start a presentation.
	7 The test system VALIDATES THAT the device A Status.Conference.Presentation.Node == Sending.
	8 The device A INVOKES API Command.Presentation.Stop() to stop a presentation.
	9 The test system VALIDATES THAT the device A Status.Conference.Presentation.Node == Off.

Property	Value
api	Command.Dial(the device B) to make a call
subject	The device A

(Part A)

CallBehavior

Test Case Specification	
Name	CallBehavior
Brief Description	The test specification aims to test the behavior of the system under test makes calls to other endpoint.
Precondition (Test Data Specification)	The device A Configuration.NetworkServices.H323.Mode=On The device A Configuration.Conference.AutoAnswer.Mode=On The device B Configuration.NetworkServices.H323.Mode=On The device B Configuration.Conference.AutoAnswer.Mode=On
Tester	None
Dependency	None

Test Setup	Name	Description
▼	Create Device	This test setup aims to create an device to start a call behavior.

Basic Flow (Test Sequence)	Steps
(Unlabeled) ▼	1 The test system creates a device as A (IP=192.168.0.1,name=spa,username=admin,password=password,rootpassword=password).
	2 The test system creates a device as B (IP=192.168.0.2,name=sba,username=admin,password=password,rootpassword=password).
	3 Postcondition The device A has been created.The device B has been created.
	4 DO
	5 UNTIL the device A Status.SystemUnit.State.NumberOfActiveCalls > 0
	6 The device A INVOKES API Command.Presentation.Start() to start a presentation.
	7 The test system VALIDATES THAT the device A Status.Conference.Presentation.Node == Sending.
	8 The device A INVOKES API Command.Presentation.Stop() to stop a presentation.
	9 The test system VALIDATES THAT the device A Status.Conference.Presentation.Node == Off.
	10 The device A INVOKES API Command.Call.DisconnectAll() to disconnect all endpoints.
	Postcondition The device A Status.Conference.Presentation.Node == Off (Test Oracle) The device A Status.SystemUnit.State.NumberOfActiveCalls == 0

Property	Value
condition	the recent called number is in the call history
subject	Tester

(Part B)

Figure 3. TCMeta instance of CallBehavior (Partial)

structuring flows of events and their relationships, together with control construct keywords (e.g., DO-UNTIL), it is then very straightforward to derive test cases by applying different testing coverage criteria (Section 5.3).

TestDataSpecification specifies test data “created or selected to satisfy the input requirements for executing one or more test cases” (from the international ISO/IEC/IEEE 29119 standard [20]). In RTCM, test data specification is provided as the precondition of a TCS. *TestOracle* is defined in the post condition of each flow of events of a TCS. A test engineer/test system uses test oracles to manually/automatically determine whether a test case has passed or failed.

Figure 2 presents the TCSs for *CallBehavior*. The TCS of *CallBehavior* refers to a test setup named as *CreateDevices*. The test setup steps make sure that the test item and the other connected systems are in the required state before starting the test cases. As one can see from the test setup description, to test the behavior of making a call, there is more than one device (VCSs) in the test environment. The basic flow of TCS of *CallBehavior* (Figure 2) has ten steps. It is important to notice that there are two specific alternative flows (TS1 and TS2) branching from the first two steps to make sure that the device in the test environment should currently have no active calls and no presentations. These two alternative flows handle the situations when there is either an active call on any of the Endpoints or any of the Endpoints is in the presentation. The original RUCM keywords such as *VALIDATES THAT* and *RESUME STEP* are reused in RTCM and also the concept of separating alternative flows from the basic flow is inherited by RTCM.

As shown in Figure 2, the *CallBehavior* TCS has two *OracleVerificationFlows* (O1 and O2), which are described as specific alternative flows. These oracle verification flows refer back to one or more steps in the test sequence, indicating which test step an oracle verification flow should verify. This specialized alternative flow provides a means to specify test oracles and connect them to specific test sequence steps such that one can easily understand what is expected after executing a TCS step. Test oracles can be also specified in postconditions of flows of events. For example, the postcondition of the basic flow of *CallBehavior* specifies that after executing the test sequence, the number of active calls of all the Endpoints is 0 and all the Endpoints are in the presentation *Off* mode.

4.2 Keywords

In addition to the RUCM keywords, RTCM introduces four new keywords: *INCLUDE TEST CASE SPECIFICATION*, *INVOKES API*, *VERIFIES THAT*, and *FOR EACH-END FOR*.

Keyword *INCLUDE TEST CASE SPECIFICATION* is defined to specify that a TCS can refer to another one to facilitate the reuse of specification fragments. This keyword, from the tooling perspective, corresponds to the *Include* semantics of use case diagrams and can therefore be formalized in *TCMeta* to facilitate automated transformation to test cases.

Keyword *INVOKES API* is particularly defined for RTCM to facilitate the generation of automatically executable test cases.

Followed by the keyword, an API of the test item is invoked to execute the test sequence step defined in the sentence. For example, as shown in step 4 of the basic flow of TCS *CallBehavior* (Figure 2), this keyword is used to invoke API *Dial* to make a call to Endpoint B. Subjects of sentences containing this keyword should always be the test system as it uses test cases to invoke APIs of the test item. Notice that this keyword is not used for specifying TCSs that are the input to derive manual test cases. But it is very important to specify TCSs that are for generating automatically executable test cases.

We define keyword *VERIFIES THAT* to provide a way for test engineers to manually verify a test sequence step and therefore subjects of sentences with this keyword must be Tester. This keyword will be used during the generation of test cases as an indication of manual steps that have to be performed. Sentences with this keyword only appear in *OracleVerificationFlows*. One example of applying this keyword can be found in O2 of the *CallBehavior* TCS (Figure 2).

4.3 Modeling Guidelines

RTCM can be used at least in the following three ways to support various testing activities: 1) Specify high-level TCSs as documentation to enable effective communication among stakeholders. Some of the features of RTCM (e.g., *InvokeAPI*) might not be useful for this purpose, which simplifies the application of RTCM in this context. Moreover, the NL aspect of RTCM eases its adoption in practice. Automated generation of high-level TCSs from requirements is also feasible, as discussed in our previous work [37]. 2) Specify low-level TCSs to enable automated generation of manually executable test cases. In this context, it is often not needed to use keyword *INVOKE API*. Keyword *VERIFY THAT* should be frequently used to specify what a tester should manually check at a specific point of time. Such low-level TCSs can of course be used for the communication purpose. Considering that RTCM is structured and restricted, it is inherently more precise than free text. Notice that in certain context (e.g., the environment stress testing in the avionics domain), test cases can only be manually executed by test engineers and a precise documentation of testing steps in an easy-to-understand manner is therefore very import RTCM can be considered as a modeling methodology to enable MBT.

5. TRANSFORMATION USING aTOUCAN4Test

Figure 4 shows an overview of the transformation from RTCM to automatically executable test cases using aToucan4Test. In the section below, we discuss its main components

5.1 RTCM Editor

TCSs are specified in the RTCM editor implemented in our modeling framework called Lightweight Modeling Framework

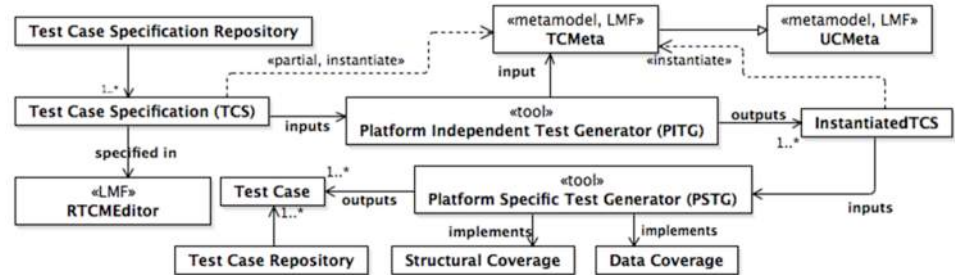


Figure 4. An overview of aToucan4Test

(LMF [36])). The framework implements similar kinds of functionalities as Eclipse Modeling Framework (EMF), but with a lightweight design with the aim to reduce tight coupling with Eclipse to facilitate easier transformations to other platforms.

Both LMF and EMF have two editors: reflective model editor and metamodel editor. LMF Reflective Editor is a simple model the LMF metamodel reflection mechanism. The LMF metamodel editor allows users to implement UCMeta extensions easily (e.g., TCMeta). The editor can also automatically generate Java code for the newly introduced metamodel or extension. We utilized this facility of LMF to implement TCMeta.

Based on the LMF framework, we implemented RTCMEditor and it was used to conduct the case studies. Keywords can be automatically highlighted and auto fill of API, status and configuration of the test item is also supported. Moreover, we embedded domain specific APIs, state variables, and configuration parameters relevant to our current industrial partner to ease specifying TCSs. However, this is a specific feature that we enabled in this industrial case study and can be disabled for other industrial applications and thus doesn't affect the generalization of the approach to other contexts. Notice that this feature can also be disabled if the RTCM editor is used to specify TCSs not targeting for the automated generation of automatically executable test cases.

5.2 Platform Independent Test Generator (PITG)

```
def test_case_12(self):
    from utility import Translate
    from lxml.etree import tostringing

(1) self.assertFalse(self.A.cuil.status.Conference.Presentation.Mode.text == "Off")
    old_status = None
    old_status = copy.copy(self.A.cuil.get_xml("/Status"))
    Translate.TranslateA("Command.Presentation.Stop()", self.A.cuil).translate()
    # Resume 2
    self.assertTrue(self.A.cuil.status.Conference.Presentation.Mode.text == "Off")
    self.assertFalse(int(self.A.cuil.status.SystemUnit.State.NumberOfActiveCalls.text) == 0)
    old_status = None
    old_status = copy.copy(self.A.cuil.get_xml("/Status"))
(2) Translate.TranslateA("Command.Call.DisconnectAll()", self.A.cuil).translate()
    # Resume 3
    self.assertTrue(int(self.A.cuil.status.SystemUnit.State.NumberOfActiveCalls.text) == 0)
    while True:
        old_status = None
        old_status = copy.copy(self.A.cuil.get_xml("/Status"))
        self.A.dial(self.B)
        if (int(self.A.cuil.status.SystemUnit.State.NumberOfActiveCalls.text) > 0): break
    old_status = None
    old_status = copy.copy(self.A.cuil.get_xml("/Status"))
    Translate.TranslateA("Command.Presentation.Start()", self.A.cuil).translate()
    self.assertFalse(self.A.cuil.status.Conference.Presentation.Mode.text == "Sending")
    old_status = None
    old_status = copy.copy(self.A.cuil.get_xml("/Status"))
    Translate.TranslateA("Command.Presentation.Start()", self.A.cuil).translate()
    # Resume 8
    self.assertTrue(self.A.cuil.status.Conference.Presentation.Mode.text == "Sending")
    old_status = None
    old_status = copy.copy(self.A.cuil.get_xml("/Status"))
    Translate.TranslateA("Command.Presentation.Stop()", self.A.cuil).translate()
    self.assertFalse(self.A.cuil.status.Conference.Presentation.Mode.text == "Off")
    old_status = None
    old_status = copy.copy(self.A.cuil.get_xml("/Status"))
    Translate.TranslateA("Command.Presentation.Stop()", self.A.cuil).translate()
    # Resume 10
    self.assertTrue(self.A.cuil.status.Conference.Presentation.Mode.text == "Off")
    old_status = None
    old_status = copy.copy(self.A.cuil.get_xml("/Status"))
    Translate.TranslateA("Command.Call.DisconnectAll()", self.A.cuil).translate()
    self.assertTrue(self.A.cuil.status.Conference.Presentation.Mode.text == "Off")
    self.assertTrue(int(self.A.cuil.status.SystemUnit.State.NumberOfActiveCalls.text) == 0)
```

Figure 5. One generated executable test case for *CallBehavior* (Partial)

The main functionality of this transformation is to instantiate TCSs written in RTCMEditor into instances of TCMeta such that transformation to executable test cases can be facilitated in a specific test scripting language. The partial instantiation of to facilitate cost-effective manual testing. 3) Specify low-level TCSs to enable automated generation of automatically executable test cases. To do so, TCSs should have the API (or any other way of providing stimulus to SUT) and test data information precisely captured. APIs of the SUT should be designed as part of the editor to enhance the applicability of the methodology, as we discussed in Section 4.2. In this context, TCMeta automatically takes place while specifying a TCS in RTCMEditor; however the part of TCMeta related to keywords is not instantiated instantly while using RTCMEditor. This part of the instantiation is however performed by PITG. PITG takes partial instantiated TCSs and TCMeta as the input and produces fully instantiated instances of TCMeta (InstantiatedTCS).

For example, as shown in Figure 2 the basic flow of TCS *CallBehavior* has 10 steps. The step 4 is “The device A INVOKES API *Command.Dial* (the device B) to make a call.” This sentence is transformed by PITG into several elements including subject “The device A” and an instance of the InvokeAPI special sentence (highlighted in Figure 3, Part A). This special sentence is further parsed to identify the API of *Command.Dial* (*Number*[192.168.0.2]), based on the information provided in the test setup (Figure 1). Another example is presented in Figure 3, Part B. The oracle verification flow O2 is automatically transformed into a set of model

elements including *rfsSentence* (the sentence containing keyword RFS), one step containing keyword VERIFIES THAT, which is further decomposed into a special sentence with two properties defined: *condition* and *subject*.

5.3 Platform Specific Test Generator (PSTG)

As shown in Figure 4, PSTG takes input *InstantiatedTCS* and outputs executable test cases, based on various coverage criteria on RTCM specifications (e.g., structural coverage and data coverage). For example, the instance of the InvokeAPI sentence is transformed into a statement in Python as shown in place (2) in Figure 5, which invokes the actual API of the VCS, i.e., *DisconnectAll*, when executed. The VALIDATES THAT sentence: step 1 of the basic flow of the TCS *CallBehavior* (Figure 2) is transformed into an assertion in Python (place (1) in Figure 5). The statement compares the actual state of a VCS with the one specified in the specification. In this example, we check whether the presentation mode of a VCS is *Off*.

The coverage criteria implemented in PSTG inspired from traditional testing coverage criteria [12; 23]. We defined two types of coverage criteria: Structural Coverage and Data Coverage. Structural coverage criteria focus on covering certain structural features of RTCM specifications. We implemented these criteria: 1) All Branch Coverage: This coverage criterion generates a set of test cases that covers all branches of RTCM specifications at least once;

2) All Condition Coverage: This coverage criterion ensures that all conditions of all branches are covered at least once. For example, for TCS *CallBehavior*, according to the All Branch Coverage and All Condition Coverage criteria, our tool generated 48 and 16 test cases, respectively; 3) Loop Coverage: This coverage criterion ensures that each loop (do until) is exercised exactly one, none, and x number of times, where x can be specified by a user beforehand. Data coverage criteria focus on covering input parameters of APIs or stimulus to a SUT in RTCM specifications. We implemented Equivalence Partitioning together with Boundary values [23]. This criterion divides the domain of an input parameter into two or more equivalence partitions and then boundary values are picked from each partition. For example, if an input parameter is restricted as $x \geq 1$ and $x \leq 10$, then we have three partitions, $x < 1$, $x \geq 1$ and $x \leq 10$, and $x > 10$. For the first partition, we select 0 and -5 (any number less than 0), for second partition we select 1, 5, 10, and for the third partition we select 11 and 20 (any number greater than 11).

6. EVALUATION

The industrial case studies are reported in Section 6.1, followed by the discussion on the applicability of RTCM (Section 6.2), the evaluation of aToucan4Test (Section 6.3). We also discuss effort required to apply our approach in Section 6.4.

6.1 Industrial Case Studies

Two large-scale cyber physical systems, respectively from the communication domain and the maritime and energy sectors, are used to evaluate our approach. One is Cisco's VCS and the other is a Subsea Oil&Gas Production System (SOGPS). Table 1 presents the characteristics of their RTCM models.

The VCS case study is from the tele-presence and communication domain and is about commercial embedded VCSs developed by Cisco Systems Norway. We focused on one such VCS product called Saturn [2], which is one of the high-end VCS developed, by Cisco. The core functionality of Saturn is already described in Section II.C. In addition, Saturn provides various functionalities related to Audio, Video, and Camera. In total, Saturn has 20 subsystems implementing these functionalities and several of these subsystems can run in parallel to each other. Saturn has on average 1 million line of C code embedded. An interested reader may consult the following reference for further details on Saturn [2]. SOGPSs are large-scale cyber physical systems managing exploitation of oil and gas production fields. SOGPS has four different types of systems, three of which are located above the sea level and the other is located in subsea. These systems have distinct functionalities and are connected through different types of communication media. Notice that the VCS case study supports automated testing in the sense that test cases are automatically executed, which is however not the case for the SOGPS case study as we reported in [19].

6.2 Applicability of RTCM

RTCM is an extension to RUCM, a use case modeling

Table 1. Characteristics of RTCM test case specifications of the two industrial case studies

Case Study		Subsystem	# TCS	# Flow of events	# Oracle Verification Flow	# Status Variable	# API Invocation
Manually Derived	VCS	Call	3	9	3	6	9
		Presentation	2	3	0	2	3
		Camera	4	7	1	5	18
		Total	9	19	4	13	30
	SOGPS	Subsea	8	3	6	N/A	N/A
		Surface	1	2	3	N/A	N/A
		Total	9	5	9	N/A	N/A
Automatically Generated	Autopilot	N/A	11	34	N/A		
	CMS		1	8			
	ATM		4	14			
	SPS		14	28			
Grand Total			48	108	13	13	30

methodology that was proposed to precisely specifying use cases for the purpose of enabling automated generation of UML analysis models [34; 35]. We have systematically evaluated the applicability of RUCM via several controlled experiments. More specifically we wanted to answer these two research questions: 1) Do users find RUCM too restrictive or impractical in certain situation? 2) Do the rules and template have a positive, significant impact on the quality of the constructed UML analysis models? The experiment results (reported in [35]) show that RUCM is easy to apply and the RUCM results into significant improvements over traditional approaches in terms of the quality of derived class and sequence diagrams.

To compare with RUCM, RTCM follows 1) the same way of specifying scenarios (i.e., flows of events), 2) the same way of applying keywords, though several new keywords are introduced to RTCM, and 3) the same mechanism of associating one TCS to the other (i.e., the “include” mechanism). We therefore expect the applicability of RTCM should be similar to RUCM, though we understand controlled experiments should be conducted to further confirm the hypothesis. However, conducting such experiments require significant amount of additional resource, which are fortunately under plan.

We also applied RTCM to specify TCSs for two industrial case studies: VCS and SOGPS. Characteristics of the two case studies are reported in Table 1. In the context of SOGPS, test engineers mainly use TCSs as the input to manually execute test cases. Therefore, there is no information regarding status variables and API invocations specified in these TCSs. Moreover, in our previous work [37], we have proposed an approach to automatically generate TCSs from use case models. To evaluate the applicability of RTCM, we carefully checked the automatically generated TCSs from four case studies and observed that the RTCM notations are sufficient. In total, as reported in Table 1, two industrial case studies in the avionics and oil and gas domains (i.e., Autopilot and SPS), and two case studies from the literature (i.e., CMS, ATM) were used in this evaluation. In summary, we manually specified 18 TCSs using RTCM and carefully verified 30 automatically generated RTCM TCSs of six case studies, which demonstrates that RTCM notations are sufficient for specifying various TCSs.

6.3 Evaluation of aToucan4Test

The current practice of testing Saturn (Section 6.1) involves writing test scripts in Python manually that can be submitted to a test case execution system that executes the test cases automatically. The current practice of manual test script writing has several challenges: 1) Writing these test scripts is a very time consuming and error prone activity and test engineers spend a significant amount of time in fixing the syntax of test

scripts; 2) There is no systematic way of writing these test scripts and there is no way to ensure that a certain kind of coverage is achieved; and 3) Accommodating changes is time-consuming process and requires studying each test script manually and reflecting changes in the changed test scripts that further require fixing syntax.

Due to these challenges, Cisco wants to automate this process as much as possible and improve the effectiveness of testing in terms of increasing coverage and in turn reducing the number of faults in the system at the minimum cost. In total, we modeled three subsystems of Saturn (i.e., Call, Presentation and Camera) to assess the applicability of RTCM as shown in Table 1. In total, we modeled 9 TCSs having 19 flows, 4 oracle verification steps, 13 status variables, and 30 API invocations. Table 2 shows the results of the test case generation from the 9 TCSs using various coverage criteria that we implemented in aToucan4Test. In total, we generated 246 executable test cases from the 9 test specifications using the coverage criteria.

Table 2. Results of the generation of executable test cases using various coverage criteria

Subsystem	Branch	Loop	Condition	Boundary	Equivalence Partition
Call	16	14	90	3	3
Presentation	3	0	6	2	2
Camera	13	16	27	10	41
Sub Total	32	30	123	15	46
Total	246				

The generated 246 test cases were successfully executed on two commercial software versions of one VCS. Due to minor differences in the names of APIs, the PSTG developed for the first version was modified a bit such that syntactically correct test cases can be generated for the second version of the software. After executing the test cases on these two versions of the software, we did not observe any fault. This is because these two versions of software are released commercial software and they have been thoroughly tested at Cisco. Recall that the motivation of the evaluation is to test the correctness of the transformation implemented in aToucan4Test. Based on the results, we are very confident that aToucan4Test can generate fully syntactically correct, automatically executable test cases.

6.4 Evaluation of Effort for Applying RTCM and aToucan4Test

To evaluate the effort required to apply RTCM and aToucan4Test, we calculate the average time (in minutes) for deriving an automatically executable test case using the following formula for the industrial case study:

$$Effort_{tc} = \frac{\text{Time required to specify TCSs}}{\text{Total number of test cases generated}} = \frac{120 \text{ mins}}{246} \approx 0.5 \text{ min}$$

It took us in total roughly 2 hours to write and review the nine TCSs. Time taken to generate all the 246 test cases can be omitted since it took less than 2 minutes. Therefore, we can conclude that the average effort per test case for applying RTCM and aToucan4Test is roughly 0.5 minutes. In practice, it is simply impossible to write a syntactically fully correct test case using any test script language within 0.5 minutes.

7. EXPERIENCE AND LESSONS LEARNT

All the experience and lessons learnt reported in this section are based on our more than seven-year of collaboration with Cisco on applying various MBT approaches.

7.1 Precise and Easy to Understand Specification

Using RTCM, test specifications can be specified precisely and are understandable by test engineers. As compared to the current solution that is writing test scripts manually in Python, Python is not understandable by everyone since the code of a test script is very close to implementation. TCSs in RTCM should be understandable by different stakeholders such as test managers, test engineers and developers since it combines NL and domain specific keywords for specifying test specifications.

To apply our proposed approach, test engineers need to be trained to use RTCM instead of a specific test script language such as Python (in Cisco). In terms of training, we are confident that RTCM should be easier to apply than any commonly applied test script language. However, in practice, a test engineer usually masters one or more test script language, which makes applying a test script language more favorable than learning RTCM from scratch. Considering the other benefits of applying RTCM and aToucan4Test, as we discussed previously, this minor limitation can be overcome.

7.2 Being Systematic and Automated

The current practice of testing VCSs at Cisco involves writing test scripts in Python manually by test engineers that can be submitted to a test case execution system to execute the test scripts automatically. The current practice lacks a systematic way of determining test objectives and writing test scripts and is driven by discussion among test engineers. Using RTCM, test specifications of a VCS can be specified more systematically and thoroughly and test scripts can be generated automatically based on systematic coverage criteria (Section 5.3). As we discussed in Section 6.4, by benefiting from automatically generating test cases, RTCM and aToucan4Test significantly improve the current practice in terms of reducing overall test case development cost.

We provide one example test case in Figure 5 to illustrate why aToucan4Test is systematic. The example can be derived from TCS *CallBehavior* by following the branch of specific alternative flow *TS2*, oracle verification flow *O1*, and then the rest of the steps of the basic flow. Notice that some statements, for example, first two lines (from statements) are the common statements used by all the test cases and do not have corresponding mappings in TCSs. The rest of the statements are typically one of the two kinds: 1) calling an API of a VCS; and 2) assert statements comparing the actual states of a VCS with the ones specified in the RTCM specifications. Needless to say that loops and conditions from the instances of TCMeta are transformed into equivalent loops and conditions in Python. In other examples, we can also have statements to query or set configurations of a VCS using additional statements in Python.

Also notice that aToucan4Test is able to generate 100% syntactically correct test scripts. As shown in Table 1, we modeled nine TCSs and by applying various coverage criteria we obtained in total 246 executable test cases, which were executed to ensure that all of them are syntactically correct.

7.3 Easier Maintenance and Better Reuse

Using RTCM, maintenance is much easier as compared to the current practice. Any change must be reflected in TCSs and syntactically correct test cases can be generated again. Taking our industrial case study as an example, making a change in the nine TCSs and then generating more than 200 test cases

automatically requires much less effort than modifying test cases directly. The idea is to raise the level of abstraction of specifying test cases such as distracting and irrelevant details can be avoided and therefore maintenance becomes naturally easier.

Reuse of TCSs and test cases is always preferred in industrial contexts. Maximizing the reuse of already developed artifacts is a very important mechanism to reduce overall product development cost. RTCM and aToucan4Test can be configured to apply different coverage criteria (Section 5.3). RTCM TCSs can be configured to accommodate variation points, which will further enhance reusability. We also aim to develop a configurable PSTG in the future in the context of product line engineering to further enhance reusability of our solution.

7.4 Comparing with Other MBT Approaches

RTCM and aToucan4Test were proposed with the following objectives: 1) relieve test engineers from the boring and time consuming activity of writing and debugging test scripts, 2) systematically implement testing strategies, 3) reduce the overall cost, and 4) enable effective and efficient communications among stakeholders (e.g., test managers and test engineers). Typically, MBT methodologies (e.g., [3]) are developed to achieve objectives 1-3, but never to achieve the fourth objective. This is because most of MBT approaches rely on complicated modeling notations e.g., UML state machines, to enable test case generation. To compare with the, RTCM is a restricted NL based modeling approach. Therefore, we expect that RTCM is easier to apply than other modeling notations enabling MBT and at the same time facilitates the effective communication between the stakeholders. We expect that RTCM will lead to low training cost and test case development cost as compared to a MBT approach; however such claims warrant more controlled experiments to provide sufficient evidence.

While working on MBT in our previous work [3] at Cisco, we realize the difficulty of developing complex UML models such as UML state machines commonly used for supporting MBT. On the other hand, practitioners are more comfortable with writing textual specifications as compared to developing UML models. Therefore, specifying TCSs as RTCM provides an alternative for test case generation for MBT using behavioral models. However, we need to conduct more empirical studies in the future to come to a definitive conclusion.

When comparing RTCM and aToucan4Test with the existing MBT approach based on UML State machine that we developed at Cisco, we observed that cost per test case using the MBT approach is roughly 6 minutes as discussed in [4]. As we discussed in Section 6.4 the rough effort per test case using RTCM is 0.5 minute, which is even lower than the MBT approach. Given the effort per test case using the MBT and RTCM approaches are 6 minutes and 0.5 minutes respectively, these approaches can be considered beneficial if the manual identification and writing of test cases costs, on average, is less than 6 minutes and 0.5 minute per test case, respectively. Based on the above effort numbers per test case, under realistic assumptions, both the MBT and RTCM approaches are clearly likely to be beneficial in many contexts as devising test cases and writing test scripts is likely to take much more time than 6 and 0.5 minute (this being probably an overestimate) in practice. Furthermore, this analysis does not account for the fact that MBT and RTCM yield test cases that are more systematic, less error-prone, and that changes to the SUT behavior can simply be

addressed by re-generating the test cases, thus facilitating the evolution of test suites. Ideally, a controlled experiment involving the manual writing of test cases would be warranted to demonstrate tangible benefits, but writing so many test cases manually is by all practical means out of reach, especially in the context of an experiment in a real industrial setting.

7.5 Comparing with Another RUCM-Based Test Generation Approach

Wang et al. [33] currently worked on another RUCM-based approach to automatically generate test cases from use case models. RTCM distinguishes itself from their work from the following aspects. 1) RTCM takes natural language based TCSs as input and automatically generates automatically executable test cases; their approach instead needs a domain model (in UML class diagram) in addition to RUCM models as input. Note that our previous work UCMeta and aToucan [35] facilitates the automated generation of such a domain model from RUCM models. 2) RTCM has embedded a systematic, configurable test data generation strategy; their approach instead requires users to specify OCL constraints manually and solve them with a constraint solver in order to generate test data. 3) RTCM has domain specific APIs, state variables, and configuration parameters embedded as part of the tool to minimize the effort required from users; their approach however requires users to manually establishing the mapping between high-level operations and actually driver function calls.

In summary, their approach contains 10 steps, five of which requires activities that have to be performed manually by software engineers. RTCM however does not require any user involvement during the test case generation process. Lastly, effort required for RTCM to generate one test case is on average 0.5 minute (including effort for specifying TCSs using RTCM). With their approach, as stated in the paper, generating one test case on average requires 12 minutes, where the OCL constraint solver [33] took most of the time. It is important to point it out that the original implementation of the OCL constraint solver (named as EsOCL) [5] took on average 3.8 minutes. Therefore, extra 8.2 minutes is most probably due to their own implementation of EsOCL. Note that our implementation of EsOCL [5] has been significantly improved to solve constraints in on average 9 seconds.

8. CONCLUSION

Test case specifications (TCSs) document a set of test cases for a SUT and are commonly used to manually code executable test cases in a test scripting language. TCSs are usually specified in NL and transition to executable test cases requires a significant amount of coding effort. To ease this transition, we developed a TCS language called Restricted Test Case Modeling (RTCM) to support automated generation of executable test cases from textual RTCM specifications by extending Restricted Use Case Modeling (RUCM). The applicability of RTCM was assessed by manually modeling two industrial case studies. In addition, we also modeled three subsystems of a Video Conferencing System developed by Cisco Systems, Norway and used aToucan4Test to automatically generate syntactically correct test cases in Python. These test cases were executed on two released commercial software versions to assess the correctness of the transformations implemented in aToucan4Test.

9. REFERENCES

- [1] Algayres, B., Lejeune, Y., and Hugonnet, F., 2012. GOAL: Observing SDL behaviors with Geode. *The Proceedings of SDL'95*, 223-230.
- [2] Ali, S., Briand, L.C., and Hemmati, H., 2012. Modeling robustness behavior using aspect-oriented modeling to support robustness testing of industrial systems. *Software & Systems Modeling* 11, 4, 633-670.
- [3] Ali, S. and Hemmati, H., 2014. Model-based Testing of Video Conferencing Systems: Challenges, Lessons Learnt, and Results. In *Software Testing, Verification and Validation (ICST), 2014 IEEE Seventh International Conference on IEEE*, 353-362.
- [4] Ali, S., Hemmati, H., Holt, N.E., Arisholm, E., and Briand, L.C., 2010. Model transformations as a strategy to automate model-based testing-A tool and industrial case studies. *Simula Research Laboratory, Technical Report (2010-01)*.
- [5] Ali, S., Zohaib Iqbal, M., Khalid, M., and Andrea, A., 2015. Improving the Performance of OCL Constraint Solving with Novel Heuristics for Logical Operations: A Search-Based Approach. *Empirical Software Engineering*.
- [6] Badri, M., Badri, L., and Naha, M., 2004. A use case driven testing process: Towards a formal approach based on UML collaboration diagrams. In *Formal Approaches to Software Testing Springer*, 223-235.
- [7] Barros, F.A., Neves, L., Hori, É., and Torres, D., 2011. The ucsCNL: A Controlled Natural Language for Use Case Specifications. In *Seke*, 250-253.
- [8] Beck, K., 2003. *Test-driven development: by example*. Addison-Wesley Professional.
- [9] Bittner, K., 2002. *Use case modeling*. Addison-Wesley Longman Publishing Co., Inc.
- [10] Bochmann, G.V., Petrenko, A., Bellal, O., and Maguiraga, S., 1997. Automating the process of test derivation from SDL specifications. *Sdl* 97, 22, 261-276.
- [11] Brömstrup, L. and Hogrefe, D., 1995. TESDL: experience with generating test cases from SDL specifications. In *Conformance testing methodologies and architectures for OSI protocols IEEE Computer Society Press*, 455-467.
- [12] Bruegge, B. and Dutoit, A.H., 2004. *Object-Oriented Software Engineering Using UML, Patterns and Java-(Required)*. Prentice Hall.
- [13] Chelimsky, D., Astels, D., Helmkamp, B., North, D., Dennis, Z., and Hellesoy, A., 2010. *The RSpec book: Behaviour driven development with Rspec, Cucumber, and friends*. Pragmatic Bookshelf.
- [14] Cucumber, 2014. *cucumber: Simple, human collaboration*. <https://cucumber.io/>.
- [15] Dssouli, R., Saleh, K., Aboulhamid, E., En-Nouaary, A., and Bourhfir, C., 1999. Test development for communication protocols: towards automation. *Computer Networks* 31, 17, 1835-1872.
- [16] Fitnesse, 2015. *FitNesse*. <http://fitnesse.org/FitNesse>.
- [17] Guo, J., White, J., Wang, G., Li, J., and Wang, Y., 2011. A genetic algorithm for optimized feature selection with resource constraints in software product lines. *Journal of Systems and Software* 84, 12, 2208-2221.
- [18] Hametner, R., Winkler, D., and Zoitl, A., 2012. Agile testing concepts based on keyword-driven testing for industrial automation systems. In *IECON 2012-38th Annual Conference on IEEE Industrial Electronics Society IEEE*, 3727-3732.
- [19] Hesari, S., Behjati, R., and Yue, T., 2013. Towards a systematic requirement-based test generation framework: industrial challenges and needs. In *Requirements Engineering Conference (RE), 2013 21st IEEE International IEEE*, 261-266.
- [20] Iso/Iec/Ieee, 2013. Software and systems engineering-Software testing-Part 1: Concepts and definitions. ISO/IEC/IEEE 29119-1:2013(E), Switzerland.
- [21] Little, G. and Miller, R.C., 2006. Translating keyword commands into executable code. In *Proceedings of the 19th annual ACM symposium on User interface software and technology ACM*, 135-144.
- [22] Mugridge, R. and Cunningham, W., 2005. *Fit for developing software: framework for integrated tests*. Pearson Education.
- [23] Myers, G.J., Sandler, C., and Badgett, T., 2011. *The art of software testing*. John Wiley & Sons.
- [24] Nebut, C., Fleurey, F., Le Traon, Y., and Jezequel, J., 2003. Requirements by contracts allow automated system testing. In *Software Reliability Engineering, 2003. ISSRE 2003. 14th International Symposium on IEEE*, 85-96.
- [25] North, D., 2006. Introducing behaviour driven development. *Better Software*.
- [26] North, D., 2007. *Introducing RBehave*. Nov., 2014. <http://dannorth.net/2007/06/17/introducing-rbehave/>.
- [27] North, D., 2007. *What's in a story?* Nov., 2014. <http://dannorth.net/whats-in-a-story/>.
- [28] Omg, 2014. Meta Object Facility (MOF) Core Specification (Version 2.4.2).
- [29] Ryser, J. and Glinz, M., 1999. A scenario-based approach to validating and testing software systems using statecharts. In *Proc. 12th International Conference on Software and Systems Engineering and their Applications Citeseer*.
- [30] Tahat, L.H., Vaysburg, B., Korel, B., and Bader, A.J., 2001. Requirement-based automated black-box test generation. In *Computer Software and Applications Conference, 2001. COMPSAC 2001. 25th Annual International IEEE*, 489-495.
- [31] Tang, J., Cao, X., and Ma, A., 2008. Towards adaptive framework of keyword driven automation testing. In *Automation and Logistics, 2008. ICAL 2008. IEEE International Conference on IEEE*, 1631-1636.
- [32] Thummalapenta, S., Sinha, S., Singhanian, N., and Chandra, S., 2012. Automating test automation. In *Software Engineering (ICSE), 2012 34th International Conference on IEEE*, 881-891.
- [33] Wang, G., Pascucci, V., Goknil, A., Braind, L., and Lqbal, Z., 2015. Automatic Generation of System Test Cases from Use Case Specifications. In *Proceedings of the International Symposium on Software Testing and Analysis (Baltimore, Maryland 2015)*.

- [34] Yue, T., Briand, L., and Labiche, Y., 2014. aToucan: An Automated Framework to Derive UML Analysis Models from Use Case Models. *ACM Transactions on Software Engineering and Methodology*.
- [35] Yue, T., Briand, L.C., and Labiche, Y., 2013. Facilitating the transition from use case models to analysis models: Approach and experiments. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 22, 1, 5.
- [36] Zhang, G., Yue, T., Wu, J., and Ali, S., 2013. Zen-RUCM: A Tool for Supporting a Comprehensive and Extensible Use Case Modeling Framework. In *Demos/Posters /StudentResearch@ MoDELS* Citeseer, 41-45.
- [37] Zhang, M., Yue, T., Ali, S., Zhang, H., and Wu, J., 2014. A Systematic Approach to Automatically Derive Test Cases from Use Cases Specified in Restricted Natural Languages. In *System Analysis and Modeling: Models and Reusability* Springer, 142-157.